

Caffeine

Caffeine

Caffeine은 자바 기반의 초고속 **In-Memory Cache Library**로, Google Guava Cache의 후계자로 개발되었다. Spring Boot 3.x에서 공식적으로 지원하는 로컬 캐시 전략 중 가장 널리 사용되며, 뛰어난 성능과 효율성으로 주목받고 있다.

1. 주요 특징

로컬 캐시의 장점

- **JVM 메모리 기반**: 데이터를 JVM 힙 메모리에 직접 저장하여 외부 서버나 네트워크 통신이 필요 없다.
- **극도로 빠른 성능**: 메모리 접근 속도를 최대한 활용하여 나노초 단위의 빠른 응답 시간을 제공한다.
- **정교한 제어**: TTL(Time To Live, 만료 시간), 최대 용량, 제거 정책 등을 세밀하게 설정할 수 있다.
- **멀티 스레드 환경 최적화**: 동시성 제어가 뛰어나 Lock 경쟁이 거의 발생하지 않는다.

Spring Cache 추상화의 저장소 구현체 중 하나가 **Caffeine Cache**이다.

2. 빠른 성능의 비밀

Caffeine의 핵심은 **ConcurrentHashMap + Window TinyLFU 알고리즘**의 조합이다.

2.1 ConcurrentHashMap 기반 구조

- 조회, 삽입, 삭제 등 대부분의 연산을 동시성이 뛰어난 자료구조로 처리한다.
- 세그먼트 락(Segment Lock) 방식을 활용하여 전체 캐시를 잠그지 않고도 안전한 동시 접근을 보장한다.

2.2 Window TinyLFU 알고리즘

Caffeine은 캐시 교체 알고리즘으로 **Window TinyLFU(Least Frequently Used)**를 사용한다.

TinyLFU의 특징

- **효율적인 빈도 추적**: 각 키가 얼마나 자주 사용되는지 효율적으로 추적하여 제거 대상을 결정한다.
- **Bloom Filter 기반**: 확률적 데이터 구조인 Bloom Filter를 사용하여 빈도를 측정하므로 메모리 사용량이 매우 적다.
- **높은 정확도**: 제거할 항목을 결정하는 정확도가 높아 캐시 히트율을 극대화한다.
- **시간 기반 감쇠(Decay)**: 오래된 접근 기록은 점진적으로 감쇠시켜 최신 패턴을 반영한다.

3. 내부 구조

Caffeine은 캐시를 세 가지 영역으로 나누어 관리한다.

3.1 Window Cache

- **역할:** 신규 데이터가 처음 진입하는 영역이다.
- **특징:** 새로운 값이 들어오면 먼저 Window에 저장되며, 일정 횟수 이상 참조되면 다음 단계로 승격된다.
- **목적:** 일회성 데이터가 Protected 영역을 오염시키는 것을 방지한다.

3.2 Protected Segment

- **역할:** 자주 참조되는 핵심 데이터를 저장한다.
- **특징:** LFU 기반으로 관리되며, 캐시에서 가장 오래 유지되는 영역이다.
- **목적:** 높은 빈도로 사용되는 데이터를 보호하여 캐시 히트율을 높인다.

3.3 Probation Segment

- **역할:** 한 번 이상 사용되었지만 아직 핵심 데이터로 인정받지 못한 항목을 저장한다.
- **특징:** Protected에서 강등되거나 Window에서 승격된 데이터가 머무른다.
- **목적:** 접근 빈도에 따라 Protected로 재승격되거나 제거될 수 있다.

Spring Cache 추상화에서 Caffeine은 단순히 캐시 저장소 역할을 수행한다.

4. Caffeine, Guava, Redis 비교

항목	Caffeine	Redis	Guava Cache
저장 위치	JVM 내부	외부 서버	JVM 내부
속도	가장 빠름	네트워크 왕복 필요	빠름
멀티 서버 공유	불가	가능	불가
TTL 지원	지원	지원	제한적
제거 알고리즘	Window TinyLFU	LRU/LFU 등	LRU
동시성 처리	매우 우수	우수	보통
적합한 용도	단일 서버, 빠른 조회	분산 캐시	간단한 로컬 캐시

5. 사용 사례

5.1 빠른 조회 API 캐싱

- 자주 조회되는 설정 값이나 코드 테이블
- 변경 빈도가 낮은 사용자 정보
- 메뉴, 카테고리 등 자주 사용되는 옵션 목록

5.2 외부 API 호출 캐싱

- 날씨 정보, 환율 데이터 등 외부 API 응답
- 공공 데이터나 외부 서버 요청 비용 절감
- API 호출 횟수 제한이 있는 서비스

5.3 다층 캐시 구조 (L1 Cache)

Redis와 혼합 사용하여 계층적 캐시 전략을 구성할 수 있다.

Caffeine (L1) → Redis (L2) → Database

- **1차 캐시(Caffeine)**: 가장 자주 접근하는 데이터를 JVM 메모리에 저장
- **2차 캐시(Redis)**: 서버 간 공유가 필요하거나 용량이 큰 데이터 저장
- **Database**: 캐시 미스 시 최종 데이터 소스

이 구조는 네트워크 비용을 최소화하면서도 분산 환경의 장점을 활용할 수 있다.

6. Spring의 기본 로컬 캐시로 선택된 이유

6.1 JVM 기반 최고 성능

Caffeine은 벤치마크에서 다른 자바 캐시 라이브러리를 압도하는 성능을 보여준다. 특히 읽기 집약적 워크로드에서 탁월한 성능을 발휘한다.

6.2 Guava Cache 대비 우수한 효율성

- **메모리 효율**: Window TinyLFU 알고리즘으로 더 적은 메모리로 높은 히트율을 달성한다.
- **동시성 처리**: Lock 경쟁이 거의 없어 멀티 스레드 환경에서도 성능 저하가 적다.
- **기능 개선**: TTL, 용량 제한, 통계 수집 등이 더 정교하고 효율적으로 관리된다.

6.3 Spring Cache 추상화와의 완벽한 호환

Spring Cache 표준 인터페이스를 완벽하게 구현하여 설정만으로 쉽게 통합할 수 있다. 기존 Spring Cache 어노테이션(`@Cacheable` , `@CachePut` , `@CacheEvict`)을 그대로 사용하면서 Caffeine의 성능을 활용할 수 있다.

6.4 활발한 커뮤니티와 유지보수

Google 출신 개발자가 주도하는 활발한 오픈소스 프로젝트로, 지속적인 업데이트와 성능 개선이 이루어지고 있다.

7. 간단한 설정 예시

```
@Configuration
@EnableCaching
public class CacheConfig {

    @Bean
    public CacheManager cacheManager() {
        CaffeineCacheManager cacheManager = new
CaffeineCacheManager("users", "products");
        cacheManager.setCaffeine(Caffeine.newBuilder()
            .maximumSize(1000)
            .expireAfterWrite(10, TimeUnit.MINUTES)
            .recordStats());
        return cacheManager;
    }
}
```
